

Design and Analysis of Algorithms

Section 2: **Problems and Algorithmic Problem Solving**

Fundamentals of Algorithmic Problem Solving

- We can consider algorithms to be procedural solutions to problems.
- These procedural solutions are not answers, but they embody specific instructions for getting answers.
- These solutions make computer science distinct from other disciplines.
 - ✓ In particular, this distinguishes it from theoretical mathematics, whose practitioners are typically satisfied with just proving the existence of a solution to a problem and, possibly, investigating the solution's properties.

لا يمكننا إعتبار الخوارزميات بأنها تمثل الحلول للProblems، وإنما هي بمثابة الإجراءات (Procedural solutions) التي تستخدم لحل تلك الProblems. وتمثل هذه الإجراءات السمة البارزة التي تميز مجال علوم الحاسوب عن بقية المجالات (عليه وضح كيف تميز هذه السمة علوم الحاسوب عن مجال الرياضيات).

- The following figure depicts a sequence of steps one typically goes through in designing and analyzing an algorithm.
 - ✓ The sequence embodies a process of algorithm design and analysis

Fundamentals of Algorithmic Problem Solving

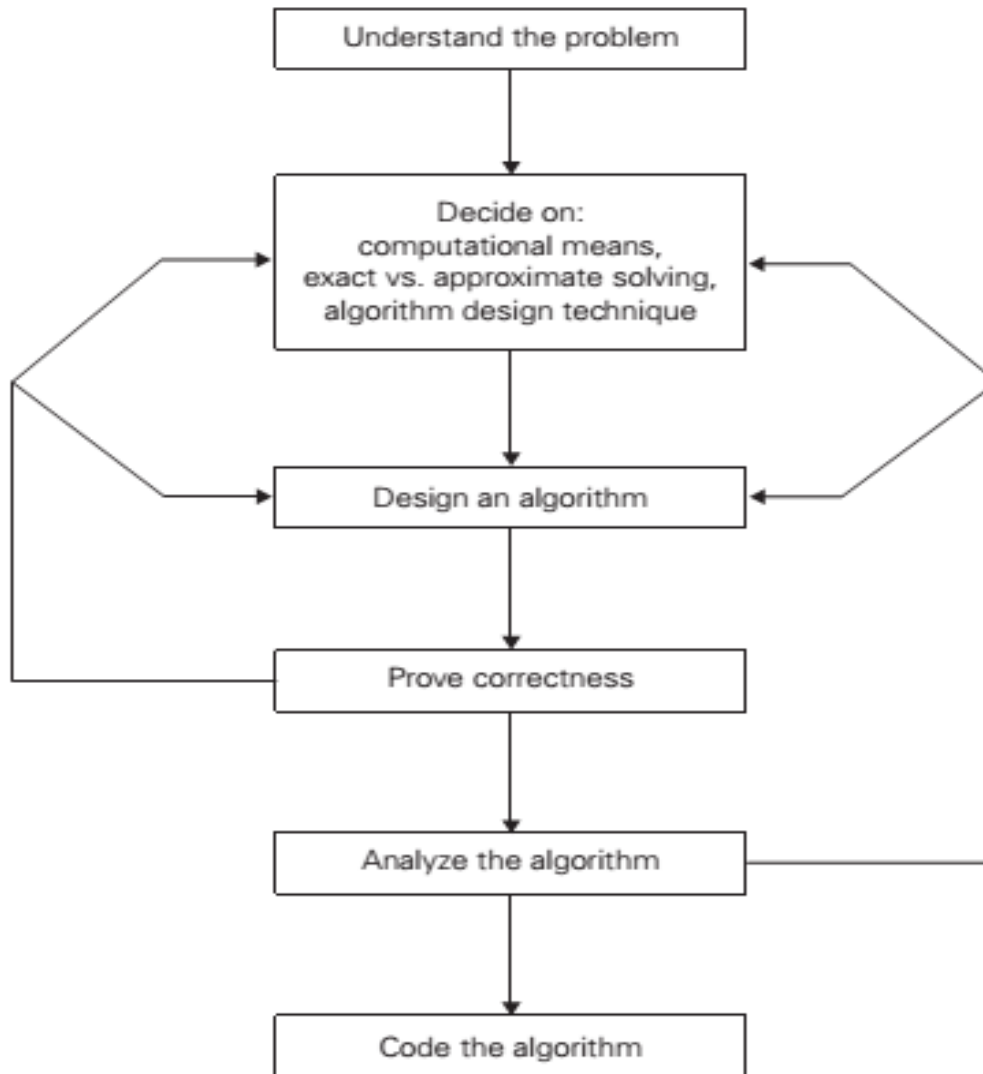


Figure 2.1 Process of algorithm design and analysis

Fundamentals of Algorithmic Problem Solving

First Two Steps of Algorithmic Problem Solving

- The algorithmic problem solving, which involves a sequence of steps, one typically goes through in providing an algorithm.
- The sequence involves nine steps which are:
 - 1) Understanding the Problem
 - ✓ Asking questions, do a few examples by hand, think about special cases, etc.
 - 2) Ascertaining the Capabilities of a Computational Device
 - ✓ This step acts to check whether the computer, which will be used, resembles the von Neumann machine, or characterizes with a parallel processing
 - ✓ Algorithms designed to be executed on von Neumann machines are called sequential algorithms
 - ✓ Algorithms which take the capabilities of the parallel processing are called parallel algorithms

Fundamentals of Algorithmic Problem Solving

3rd and 4th Steps of Algorithmic Problem Solving

- These steps are:

- 3) Choosing between Exact and Approximate Problem Solving

- ✓ An **exact algorithm** is the algorithm which solves its problem exactly, whereas an **approximate algorithm** is the algorithm that solves its problem approximately.
 - ✓ **Approximate algorithms** will be chosen for two reasons:
 - First, there are important problems that simply cannot be solved exactly, such as extracting square roots, solving nonlinear equations, and evaluating definite integrals
 - Second, available algorithms for solving a problem exactly can be unacceptable slow because of the problem's intrinsic complexity (e. g., the traveling salesman problem)

- 4) Deciding on Appropriate Data Structures

- ✓ Some algorithms do not demand any ingenuity in representing their inputs. But others are, in fact, predicated on ingenious data structures
 - ✓ Some of the algorithm design techniques depend intimately on structuring or restructuring data specifying a problem's instance

Fundamentals of Algorithmic Problem Solving

5th Step of Algorithmic Problem Solving

- The step is:

- 5) Algorithm Design

- ✓ There are methods (named as **algorithm design techniques**) that provide a guidance for designing algorithms for new problems.
 - ✓ An **algorithm design technique** (or “**strategy**” or “**paradigm**”) is a generic model or framework which underlies the design of a class of algorithms.
 - ✓ The following is a list of well-known for these techniques
 - 1) Brute force
 - 2) Divide and conquer
 - 3) Decrease and conquer
 - 4) Transform and conquer
 - 5) Greedy approach
 - 6) Dynamic programming
 - 7) Backtracking and Branch and bound
 - 8) Space and time tradeoffs

Fundamentals of Algorithmic Problem Solving

6th to 9th Steps of Algorithmic Problem Solving

- The steps are:

- 6) Methods of Specifying an Algorithm

- ✓ Natural Languages, Pseudocode, and Flowcharts are the options that are widely used nowadays for specifying algorithms

- 7) Proving an Algorithm's Correctness

- ✓ It is to prove that the algorithm yields a required result for every legitimate input in a finite amount of time.
 - ✓ A common technique for the proving is to use mathematical induction because an algorithm's iterations provide a natural sequence of steps for such proofs
 - ✓ Although, tracing the algorithm performance for a few specific inputs can be a very worthwhile, it can not prove the algorithm correctness conclusively. But in order to show that an algorithm is incorrect, you need just one instance of its input for which the algorithm fails

- 8) Analyzing an Algorithm

- 9) Coding an Algorithm

Important Problem Types

There is a limitless sea of problems those can be solved by the use of computers. The following is the list introduces some examples of these problems.

- Sorting
- Searching
- String processing
- Graph Problems
- Combinatorial Problems
- Geometric Problems
- Numeric Problems

These problems are used in subsequent sections to illustrate different algorithm design techniques and methods of algorithm analysis.

Important Problem Types

Sorting Problem (مسألة الترتيب)

- Sorting can be defined as a rearrangement of items for a given list in ascending/descending order.
 - Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
 - Output: A reordering $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.
- Why sorting?
 - Help searching
 - Algorithms often use sorting as a key subroutine.
- Sorting key
 - A specially chosen piece of information used to guide sorting. I.e., sort student records by names.

Important Problem Types

Sorting Problem (مسألة الترتيب)

- Examples of sorting algorithms
 - Insertion sort
 - Selection sort
 - Bubble sort
 - Merge sort
 - Heap sort ...

Important Problem Types

Sorting Problem (مسألة الترتيب)

Insertion Sorting Algorithm:

```
1. Procedure Insert(T[1..n], n)
2.   for i := 2 to n
3.     begin
4.       x := T[i]
5.       j := i - 1;
6.       while (j > 0) and (T[j] > x)
7.         begin
8.           T[j+1] := T[j]
9.           j := j - 1
10.        end
11.      T[j+1] := x
12.    end
13. End Insert
```

Important Problem Types

Sorting Problem (مسألة الترتيب)

Selection Sorting Algorithm:

```
1. Procedure Select(T[1..n], n)
2.   for i = 1 to n-1 do
3.     begin
4.       minj := i
5.       minx := T[i]
6.       for j:=i+1 to n do
7.         begin
8.           if T[j]<minx then
9.             begin
10.              minj:=j
11.              minx:=T[j]
12.            end
13.          end
14.        T[minj] := T[i]
15.        T[i] := minx
16.      end
17.    end Select
```

Important Problem Types

Searching Problem (مسألة البحث)

- Searching is one of the most fundamental and widely encountered problems in computer science.
- Given a **collection of objects**, the goal of search is to **find** a **particular object** in this collection *or* to **recognize** that the **object** does not exist **in the collection**.
 - ✓ Often the objects have **key** values on which one searches and data values which correspond to the information one wishes to retrieve once an object is found.
 - ✓ For example, a telephone book is a collection of names (on which one searches) and telephone numbers (which correspond to the data being sought).
- **Searching Algorithms** are designed to check or retrieve an element from any **data structure** where it is being stored.
 - Searching is one of the basic operations of Data Structure, which is used to find a particular data from a list of data.
 - Searching can operate both on sorted and unsorted list of elements.
 - Depending on the type of data structure, a suitable search algorithm is chosen.

Important Problem Types

Searching Problem (مسألة البحث)

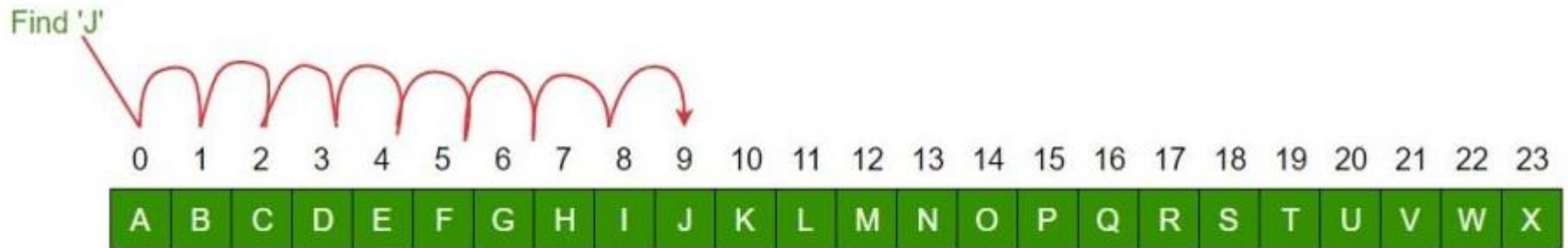
- There are so many types of search algorithms used to search an element from a set of elements.
- Search algorithm includes both:
 - 1) Traditional search algorithm (i.e. which are used very frequently)
 - 2) Proposed search algorithms (i.e. the search algorithms which are recently proposed by the researcher)
- **Traditional search algorithms** are like **linear search, binary search, interpolation search** and **jump search**.
- Some types of Searching Algorithms:
 1. Linear Search
 2. Binary Search
 3. Jump Search
 4. Interpolation Search
 5. Exponential Search
 6. Sublist Search (Search a linked list in another list)
 7. Fibonacci Search
 8. The Ubiquitous Binary Search

Important Problem Types

Searching Problem (مسألة البحث)

Sequential Search Algorithm:

- A linear search or sequential search is a method for finding an element within a list.
- This type of searching algorithms sequentially checks each element of the list until a match is found or the whole list has been searched.
- A linear search runs in at worst linear time and makes at most n comparisons, where n is the length of the list.



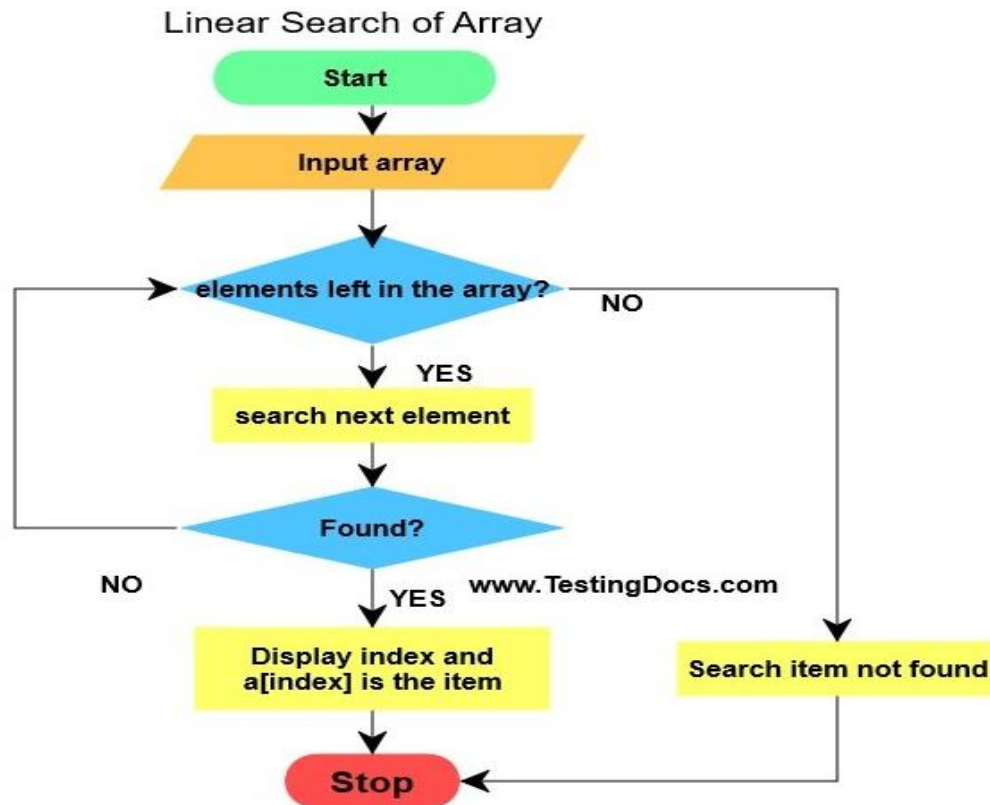
- If each element is equally likely to be searched, then linear search has an average case of $n + 1/2$ comparisons, but the average case can be affected if the search probabilities for each element vary.

Important Problem Types

Searching Problem (مسألة البحث)

Sequential Search Algorithm:

- Linear search is rarely practical because other search algorithms and schemes, such as the binary search algorithm and hash tables, allow significantly faster searching for all but short lists.



Important Problem Types

Searching Problem (مسألة البحث)

Sequential Search Algorithm:

The pseudocode of the binary search algorithm can be presented as follows.

```
1. Procedure Sequential_Search( $A[0..n-1]$ ,  $K$ )
2.    $i := 0$ 
3.   while  $i < n$  and  $A[i] \neq K$  do
4.      $i := i + 1$ 
5.   if  $i < n$ 
6.     return ( $i$ )
7.   else
8.     return (-1)
9. end Sequential_Search
```

Important Problem Types

Searching Problem (مسألة البحث)

Binary Search Algorithm

- A binary search is an advanced type of search algorithm that finds and fetches data from a sorted list of items.
- Its core working principle involves dividing the data in the list to half until the required value is located and displayed to the user in the search result.
- Binary search is commonly known as a **half-interval search** or a **logarithmic search**.
- The binary search works in the following manner:
 - ✓ The search process initiates by locating the middle element of the sorted array of data
 - ✓ After that, the key value is compared with the element
 - ✓ If the key value is smaller than the middle element, then searches analyses the upper values to the middle element for comparison and matching
 - ✓ In case the key value is greater than the middle element then searches analyses the lower values to the middle element for comparison and matching
- See the following example

Important Problem Types

Searching Problem (مسألة البحث)

Binary Search Algorithm

Find 9

3 5 7 8 9 12 15

Important Problem Types

Searching Problem (مسألة البحث)

Binary Search Algorithm

Find 9

3

5

7

8

9

12

15

Important Problem Types

Searching Problem (مسألة البحث)

Binary Search Algorithm

Find 9

3

5

7

8

9

12

15

Important Problem Types

Searching Problem (مسألة البحث)

Binary Search Algorithm

Find 9

3

5

7

8

9

12

15

Important Problem Types

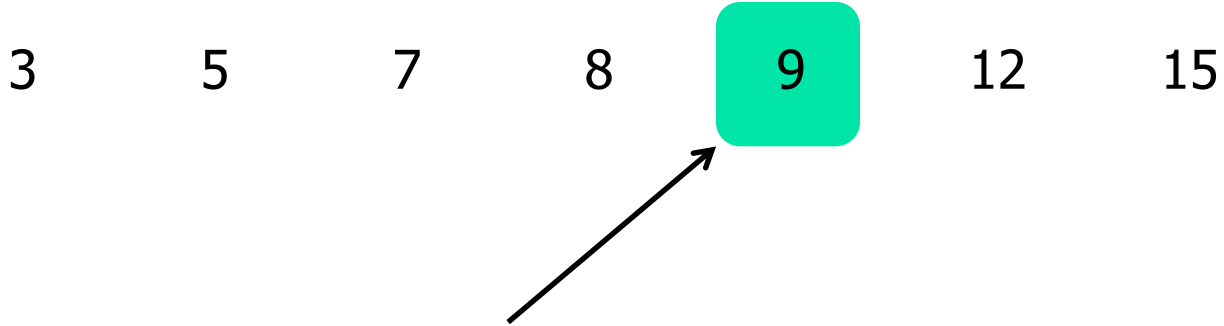
Searching Problem (مسألة البحث)

Binary Search Algorithm

Find 9

3 5 7 8 9 12 15

It is found



Important Problem Types

Searching Problem (مسألة البحث)

Binary Search Algorithm

The pseudocode of the binary search algorithm can be presented as follows.

```
1. Procedure Binary_Search ( $A[1..n]$ ,  $K$ )
2.   $L := 1$ 
3.   $R := n$ 
4.  while  $L \geq R$  do
5.      begin
6.           $m := \text{ceil}((R - L)/2)$ 
7.          if  $K = A[m]$  return ( $m$ )
8.          else
9.              if  $K < A[m]$   $R := m$ 
10.             else  $L := m$ 
11.         end
12. return (-1)
13. end Binary_Search
```


Important Problem Types

String Processing Problems (مسائل معالجة السلاسل)

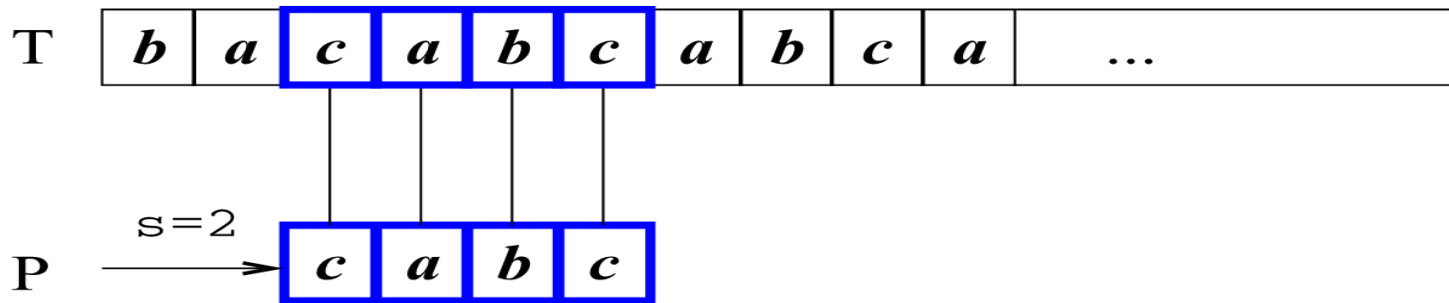
- In recent decades, the rapid proliferation of applications dealing with **non-numerical data** has intensified the interest of researchers and computing practitioners in **string-handling algorithms**.
- A **string** is a sequence of characters from an alphabet.
- String of a particular interest are:
 - 1) **text strings**, which comprise letters, numbers, and special characters.
 - 2) **bit strings**, which comprise zeros and ones
 - 3) **gene sequences**, which can be modeled by strings of characters from the four-character alphabets {A, C, G, T}.
- **String-processing algorithms** have been important for computer science for a long time in conjunction with computer languages and compiling issues.
- **String matching problem (مسألة التحقق من تطابق السلاسل)** is one particular string processing problem—that of searching for a given word in a text—has attracted special attention from researchers.

Important Problem Types

String Processing Problems (مسائل معالجة السلاسل)

String Marching/Searching Problem

- Given a **text** array $T[1 \dots n]$ and a **pattern** array $P[1 \dots m]$, such that the elements of T and P are characters taken from alphabet Σ (e.g., $\Sigma = \{0, 1\}$ or $\Sigma = \{a, b, \dots, z\}$)
- The problem is to find all the occurrence of P in T
 - ✓ A pattern P occurs with **shift** s in T , if $P[1 \dots m] = T[s + 1 \dots s + m]$
 - ✓ The **String Matching Problem** is to find all values of s
 - ✓ Obviously, we must have



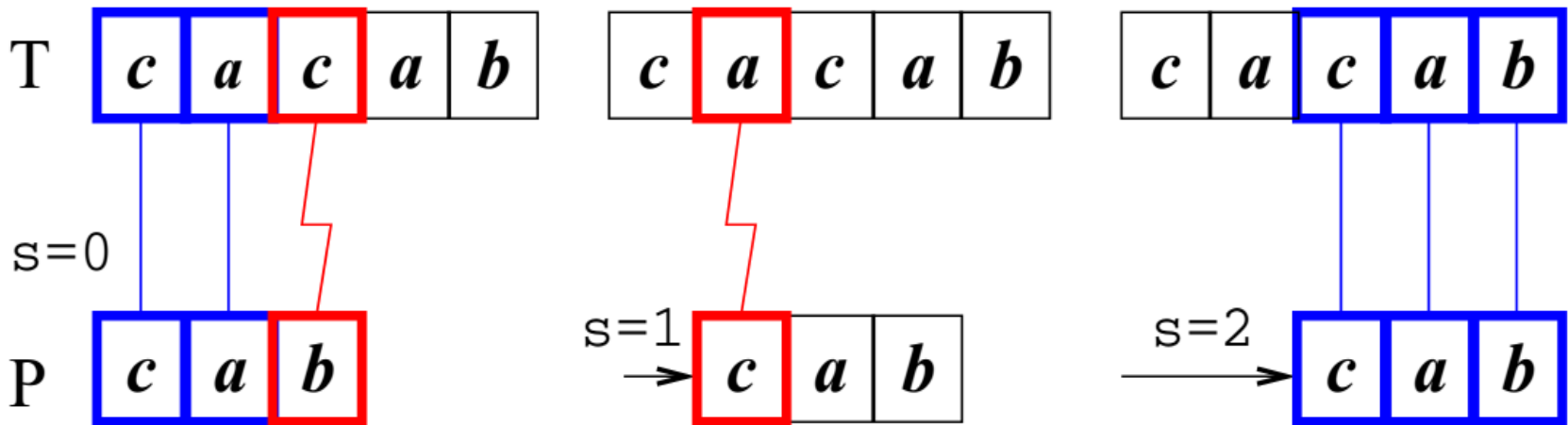
- There are a number of string searching algorithms in existence today, but the two we shall review are: **Brute Force** and **Rabin-Karp**.

Important Problem Types

String Processing Problems (مسائل معالجة السلاسل)

Brute Force Algorithm of String Matching Problem

- Initially, **P** is aligned with **T** at the first index position.
 - ✓ **P** is then compared with **T** from **left-to-right**.
 - ✓ If a mismatch occurs, "slide" **P** to right by 1 position, and start the comparison again.



Important Problem Types

String Processing Problems (مسائل معالجة السلاسل)

Brute Force Algorithm of String Matching Problem

```
BF_StringMatcher(T, P) {  
    n = length(T);  
    m = length(P);  
    // s increments by 1 in each iteration  
    // => slide P to right by 1  
    for (s=0; s<=n-m; s++) {  
        // starts the comparison of P and T again  
        i=1; j=1;  
        while (j<=m && T[s+i]==P[j]) {  
            // corresponds to compare P and T from  
            // left-to-right  
            i++; j++;  
        }  
        if (j==m+1)  
            print "Pattern occurs with shift=", s  
    }  
}
```

Important Problem Types

Graph Problems

- A graph is a collection of points called **vertices**, some of which are connected by line segments called **edges**.
- Graph can be used for modeling a wide variety of real-life applications, such as
 - Modeling WWW
 - communication networks
 - Project scheduling ...
- Examples of graph algorithms
 - Graph traversal algorithms
 - Shortest-path algorithms
 - Topological sorting

Important Problem Types

Combinatorial Problems

- These problems ask (explicitly or implicitly) to find a combinatorial object —such as a permutation, a combination, or a subset — that satisfies certain constraints and has some desired property (e.g., maximizes a value or minimizes a cost)
- Combinatorial problems are most **difficult problems** in computing. Their difficulties stems from the following facts
 - First, The number of combinatorial objects typically grows extremely fast with a problem's size, reaching unimaginable magnitudes even for moderate-sized instances.
 - Second, there are no known algorithms for solving most such problems exactly in an acceptable amount of time. Moreover, most computer scientists believe that such algorithms do not exist.

Important Problem Types

Geometric Problems

- Geometric algorithms deal with geometric objects such as points lines, and polygons.
- Ancient Greeks were very much interested in developing procedures for solving variety of geometric problems including problems of constructing simple geometric shapes (e.g., triangles, circles, and so on).
- Then, for about 2000 years , intense interest in geometric algorithms disappeared.
- Today people are interested in geometric algorithms with quite different applications such as computer graphics, robotics, and tomography.

4.8) Numeric Problems

Numeric Problems

- These are problems which involve mathematical objects of continuous nature (e.g., solving equations and systems of equations, computing definite integrals, evaluating functions, and so on).
- Many sophisticated algorithms have developed over the years in this area, and they continue to play a critical role in many scientific and engineering applications, but in the last decades the computer industry has shifted its focus to business applications which require algorithms for information storage, retrieval, transportation through networks, and presentation to users.
- As a result of this revolutionary change, numerical analysis has lost its formally dominating position in both industry and computer science.
- Nevertheless, it is important for any computer-literate person to have at least a rudimentary about numerical algorithms.